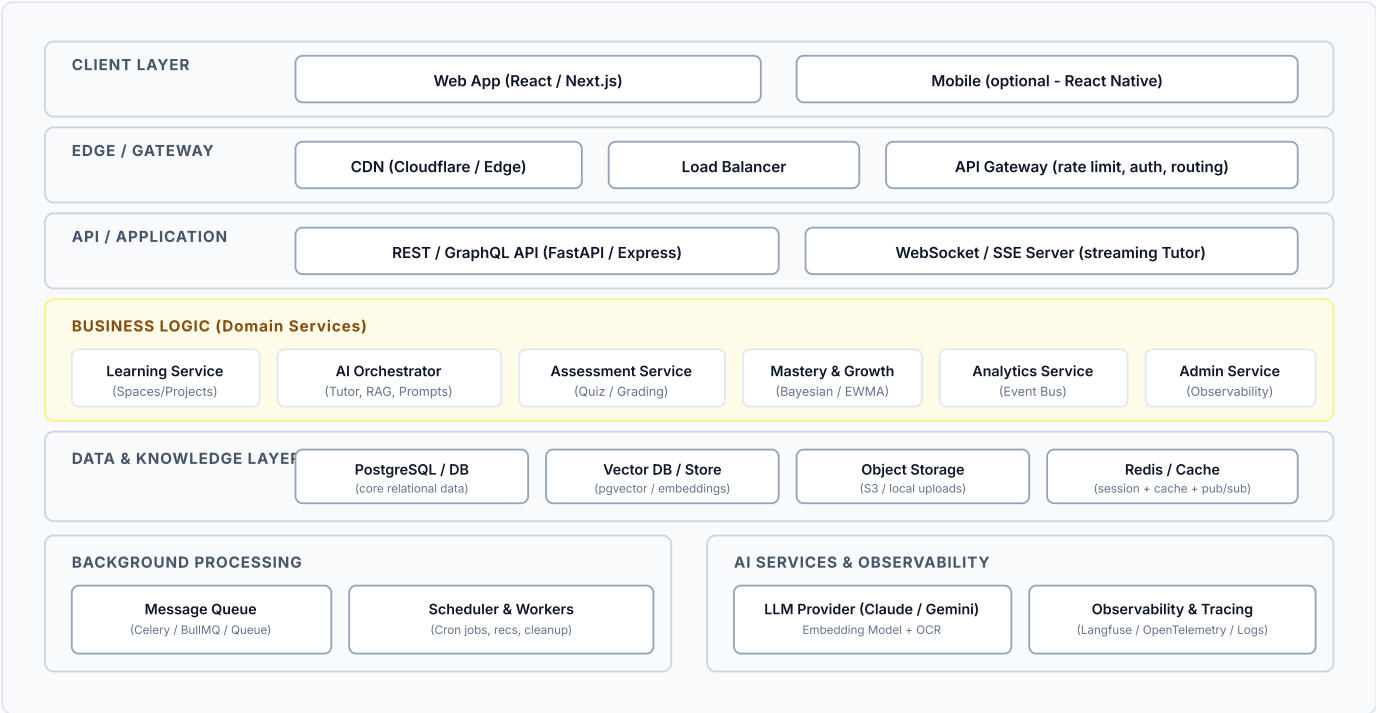


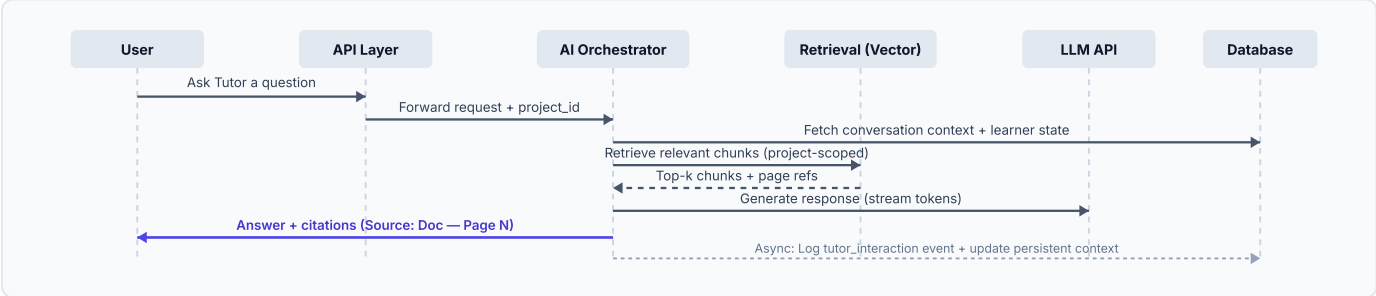
AI Study Companion — System Architecture

Real-time, production-grade architecture derived from the PRD

1. High-Level System Architecture



2. Core Learning Loop — Sequence Flow

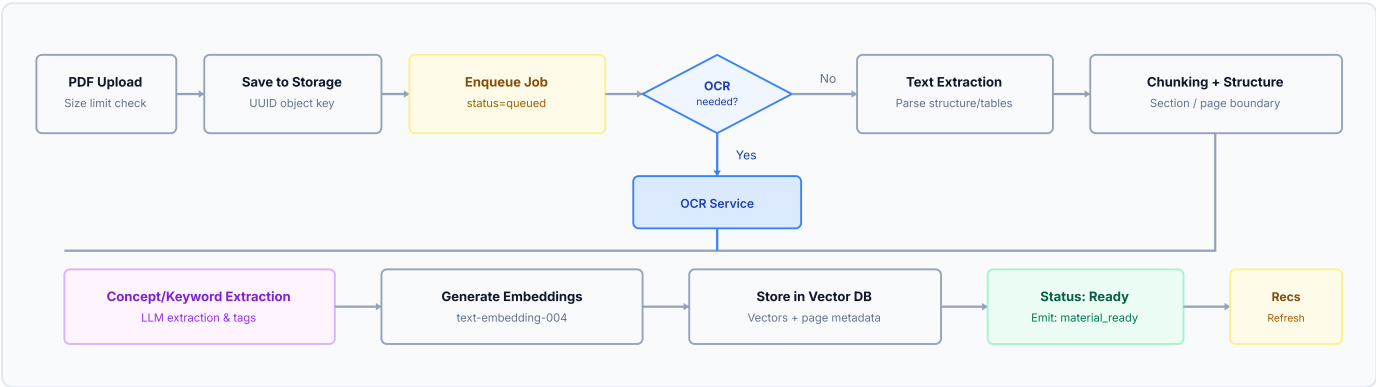


AI Study Companion — System Architecture

Asynchronous Processing & Document Understanding Pipeline

3. Document Processing Pipeline (Async)

The ingestion pipeline processes course materials asynchronously through a staged, idempotent workflow. Each material transitions sequentially from upload to active vector search index:



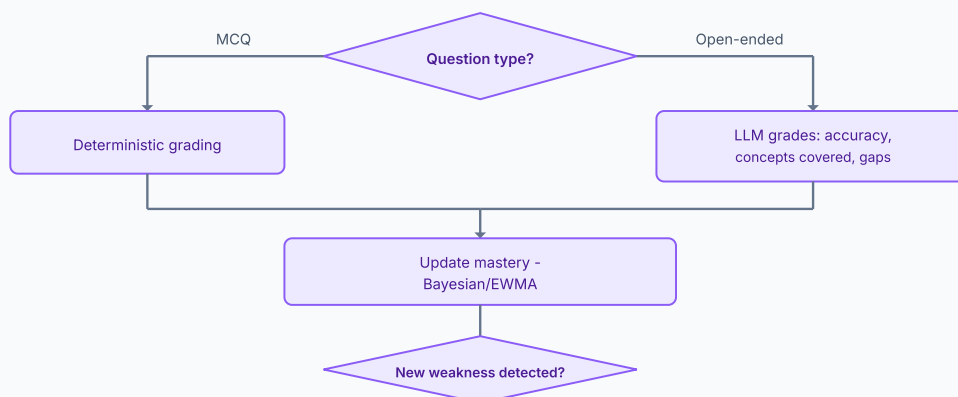
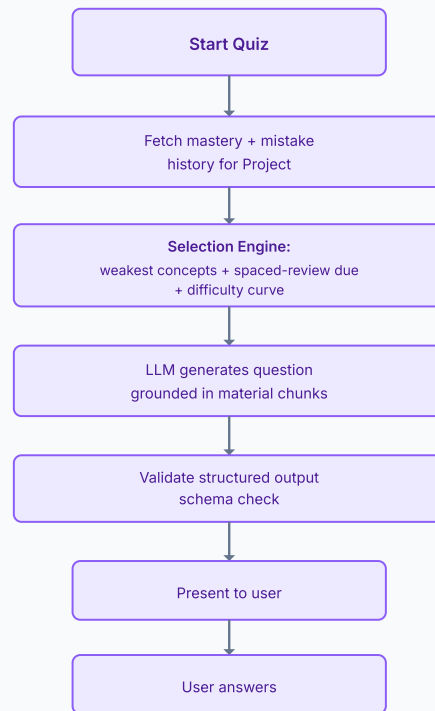
Failure handling: each stage is idempotent (checked via `job_id + material_id` dedup key), retried with exponential backoff (max 3 attempts), and on final failure the material status flips to `failed` with an error reason surfaced to the user + admin.

AI Study Companion — System Architecture

Adaptive Assessment & Knowledge Mastery Engine

4. Adaptive Quiz Flow

The assessment workflow models a continuous adaptive loop: selecting target concepts by student mastery gaps, synthesizing grounded questions, validating structured schema outputs, and recalculating mastery curves:



AI Study Companion — System Architecture

Recommended Tech Stack & Architectural Decision Justification

5. Recommended Tech Stack (justified)

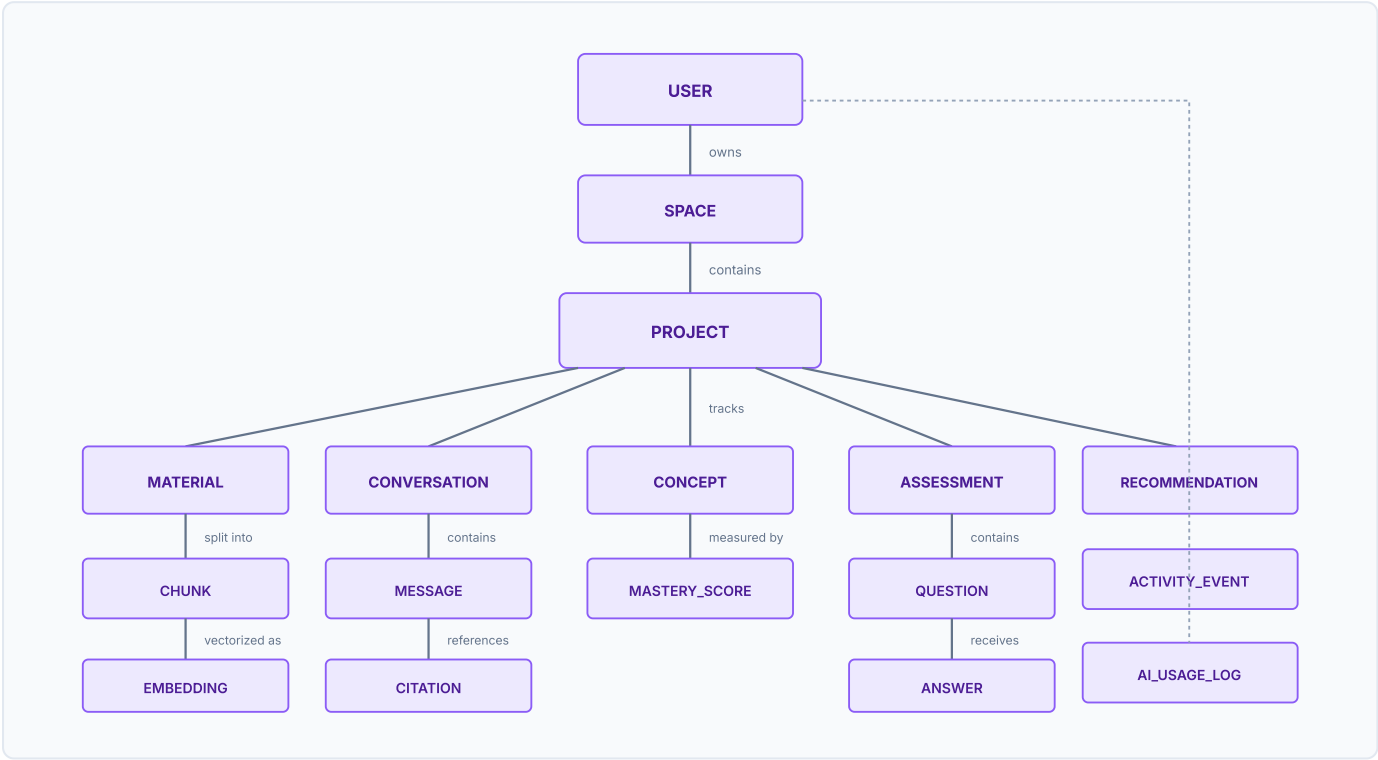
Layer	Choice	Why
Frontend	Next.js (React, App Router) + Tailwind	SSR for fast loads, streaming UI support for Tutor, huge ecosystem.
Backend API	FastAPI / Express (Node.js)	Native async, great for AI-heavy I/O-bound workloads, schema validation matches "structured AI output validation" requirement.
Realtime	WebSockets / SSE	Streaming Tutor responses per Should-Have requirements.
Database	PostgreSQL / In-Memory JSON Store	Relational integrity for Users/Spaces/Projects/Assessments; JSON for flexible AI metadata.
Vector Store	pgvector (same Postgres) / In-Memory Store	Keeps infra simple for prototype scale; avoids a second system to operate; project-scoped query time filtering.
Object Storage	S3 (or S3-compatible, Cloudflare R2)	Durable, cheap PDF storage, signed URLs for secure access.
Cache	Redis / In-Memory Cache	Session cache, rate limiting, pub/sub for job status updates.
Queue/Workers	Celery / BullMQ / BackgroundQueue	Background doc processing, mastery updates, recommendation generation.
LLM Provider	Claude API / Gemini Pro & Flash	Native tool-use, strong grounded-answer behavior, cost-tiered models (Pro for reasoning, Flash/Haiku for cheap classification).
Embeddings	Google text-embedding-004 / OpenAI	High retrieval quality, dense semantic representation.
OCR	Gemini Multimodal / Unstructured.io	Handles scanned pages, mathematical equations, tables, and diagrams.
Auth	JWT + refresh tokens (salted scrypt)	Fast to implement, secure, supports RBAC (user vs admin).
Observability	Langfuse + Structured Logs	Model-agnostic LLM tracing (prompt, latency, tokens, cost per request) satisfying AI observability & evaluation requirements.
Deployment	Vercel (frontend) + Render/Railway	Fast public deployment within prototype timeline.
CI/CD	GitHub Actions	Test + deploy pipeline, satisfies "testing + deployment" requirement.

AI Study Companion — System Architecture

Relational Domain Model & Entity Relationships

6. Data Model (Core Entities & Relationships)

The entity relationship model establishes explicit tenant ownership and strict hierarchical containment:



AI Study Companion — System Architecture

Security Architecture, Isolation & Evaluation Loops

7. Security & Isolation Model

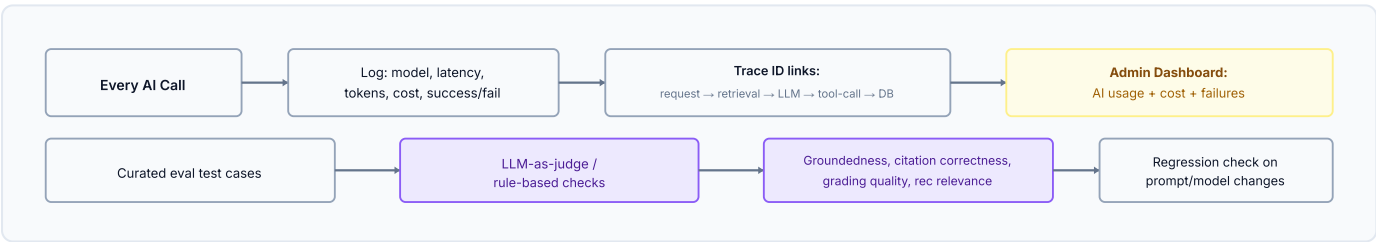
The security architecture enforces layered defense across request routing, retrieval queries, and LLM boundary containment:



Key controls:

- **Row-Level Security (Postgres RLS) / Project Scoping:** Scoped strictly by `user_id` / `project_id` — enforces isolation even against potential application query bugs.
- **Prompt-injection defense:** Material content and user messages are always passed as data fields in the LLM request (`<document>` , `<user_input>`), never concatenated into system instructions; tool-calls from the LLM are validated against a strict allowlist and schema before execution.
- **Signed, expiring URLs:** Expiring tokens for document storage access.
- **Secrets Management:** Secrets via environment variables / secret manager, never committed to git repository.

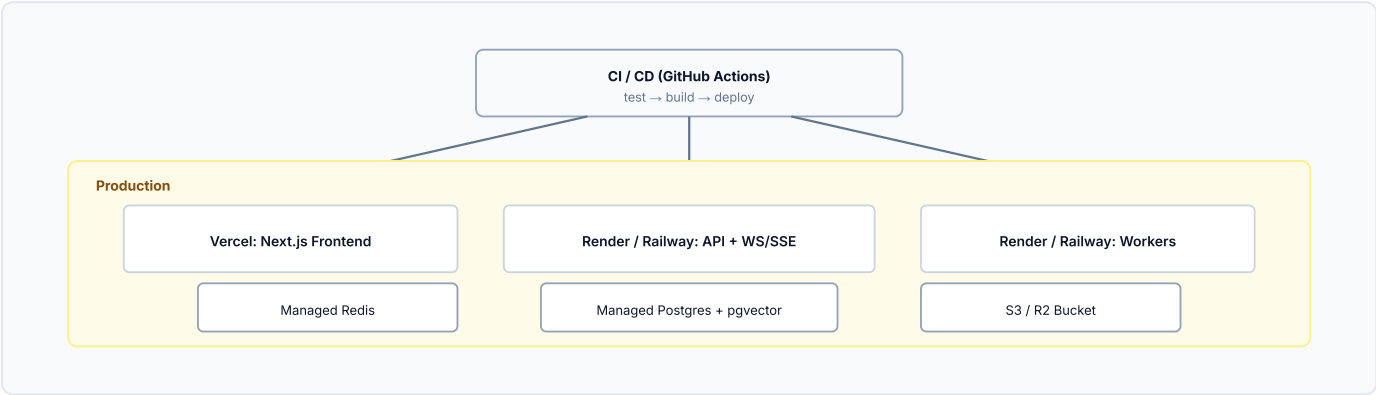
8. Observability & Evaluation Loop



AI Study Companion — System Architecture

Deployment Topology & PRD Requirements Fulfillment

9. Deployment Topology



10. Why This Architecture Satisfies the PRD

PRD Requirement	How it's satisfied
Context isolation per Project	RLS + <code>project_id</code> scoping in every query, retrieval filtered by project at query time.
Evidence over guessing	RAG pipeline returns "insufficient evidence" branch when retrieval score < threshold.
Async by design	Queue + worker pool for all long-running ops (upload, grading, recs).
Observable AI	Langfuse tracing + structured logs + cost/latency dashboard.
Safe AI interaction	Tool-calling layer with schema validation, no direct DB access from LLM.
Idempotency	Job dedup keys, event processing with unique event IDs.
Security	JWT auth, RLS, signed URLs, input validation, prompt-injection isolation.
Testing	GitHub Actions pipeline running auth/AI/mastery/background-job test suites.

This document is meant to accompany the submission's "Architecture Documentation" requirement (Section 20.4) — pair it with a written rationale of what was simplified for the 3–4 day timeline and what you'd improve given more time (e.g., moving pgvector → dedicated vector DB at scale, adding a dead-letter queue, multi-region deployment).